



ELEC3609/9609 - Week 2 Lab

02 RESTful API Design and Team Collaboration with GitHub

In this lab, you will explore key concepts in modern web development, including how APIs enable communication between frontend and backend systems, and how teams collaborate effectively using GitHub. You'll also examine software specification and architecture practices that support scalable, maintainable applications.

Individual Task: Individual Task 2 - Analyse a RESTful API

Lab Resources: API Contract template.

Learning Objectives

After completing this lab, students should be able to:

- Explain how web applications interact using REST API-based communication.
- Understand the role of architectural principles in structuring web applications.
- Recognise how backend and frontend teams coordinate using shared API contracts.
- Apply essential GitHub workflows for team collaboration and project tracking.

Chapter 1. RESTful APIs

In the late 1990s, web URLs often directly exposed server-side scripts and included explicit arguments to indicate actions and data, such as:

```
getdata.php?user=123&action=fetch
```

This approach, while functional, tightly coupled URL structures to specific implementation details, making systems less flexible and harder to maintain. It also posed security risks by

exposing methods and parameters directly in the URL.

URLs in the late 1990s

`http://uptontea.com/shopcart/toc.asp?parent=Teas&child=Green`

script

arguments

As web development evolved, the need for more scalable, secure, and maintainable solutions led to the adoption of **RESTful APIs**. RESTful architecture uses standardised HTTP methods and focuses on **stateless communication** and **resource-based URLs** (e.g., `GET /users/123`), which abstract server implementation details from the client.

This shift not only enhances security by standardising access and utilising modern security practices but also supports better scalability and flexibility, accommodating modern web practices and a wide range of client types including mobile apps and other web services.

RESTful API endpoint

GET

`http://tea.com/teas/123/reviews/4`

HTTP verb
gives the action

path names a resource



TIP

RESTful API is not a protocol, library, or tool. It is a style/convention or a set of design principles for building web APIs. REST uses HTTP as the underlying protocol, applying standardised methods like GET, POST, PUT, and DELETE to interact with resources identified by URLs.

1.1 How Does a RESTful API Work?

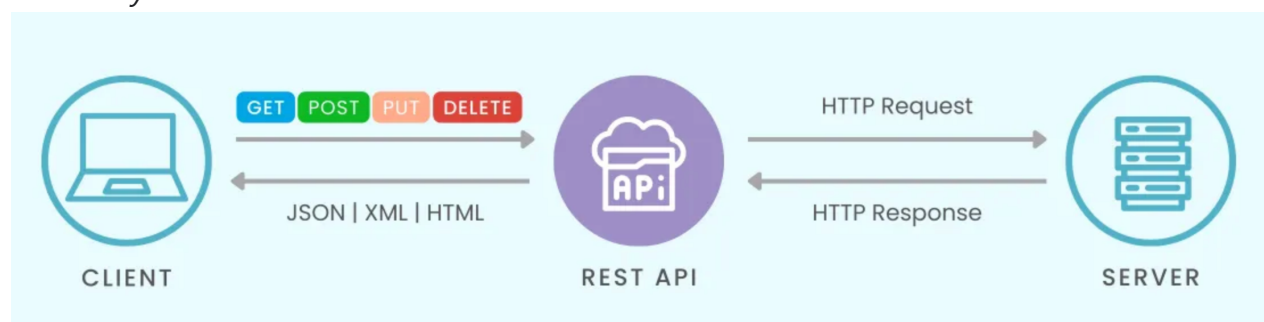
The basic function of a RESTful API is similar to browsing the internet. The client contacts the

server by making a request to retrieve or manipulate a resource. The server responds with the result.

A typical RESTful API call works through the following steps:

1. The client sends an HTTP request to the server.
2. The server authenticates the request if needed (e.g., with an API key or token).
3. The server processes the logic (e.g., fetch, create, update, delete data).
4. The server sends an HTTP response containing status and data (e.g., JSON).

A RESTful API separates client and server, allowing independent development and scalability.



1.2 RESTful API Client Request

A RESTful API request typically includes the following components:

Unique Resource Identifier

Each resource is identified by a **URL** (Uniform Resource Locator), also known as the **request endpoint**. Example:

```
GET https://api.example.com/posts/22
```

HTTP Method

The HTTP method tells the server what action to perform on the resource.

Method	Description
--------	-------------

Method	Description
GET	Retrieve data from the server. GET requests can be cached and may include query parameters.
POST	Send new data to the server. Repeating a POST may result in multiple records.
PUT	Update an existing resource. Repeating a PUT gives the same result each time.
DELETE	Remove a resource. May fail if authentication is not valid.

HTTP Headers

Headers provide **metadata** about the request and response format, authentication, and caching. Example:

```
Content-Type: application/json
Accept: application/json
Authorization: Bearer <token>
```

Request Body (Data)

For methods like **POST** and **PUT**, the client includes a **data payload** in the request body. Example:

```
{
  "title": "New Post",
  "author": "User123"
}
```

Parameters

Clients may send extra information through parameters to help the server process the request.

Parameter Type	Description
Path Parameters	Part of the URL to identify specific resources. E.g., <code>/posts/22</code>

Parameter Type	Description
Query Parameters	Provide filtering or sorting logic. E.g., <code>/posts?author=Admin</code>
Cookie Parameters	Sent via cookies, often for session or quick auth purposes.

Example `curl` Command for RESTful API Request

```
curl -X PUT https://api.imgur.com/3/account/ImgurUser/settings \
  -H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
  -H "Content-Type: multipart/form-data" \
  -F "bio=Long time lurker..." \
  -F "public_images=false" \
  -F "messaging_enabled=true" \
  -F "accepted_gallery_terms=true" \
  -F "show_mature=true" \
  -F "newsletter_subscribed=true"
```

Part	Explanation
<code>-X PUT</code>	Specifies the HTTP method as <code>PUT</code> , which updates a resource.
<code>-H</code>	Adds a custom HTTP header. In this example, two headers are used:
<code>Authorization: Bearer ...</code>	Passes the access token to authenticate the request.
<code>Content-Type: multipart/form-data</code>	Informs the server you're sending form fields with possibly file-like format.
<code>-F</code>	Sends a form field as key-value pair. Multiple <code>-F</code> flags are used to send multiple fields in the body.

1.3 RESTful API Authentication Methods

A RESTful web service authenticates requests before sending responses. Authentication verifies the client's identity and ensures only authorised users can access protected resources.

Common Authentication Methods

Method	Description	Security Notes
Basic Authentication	Sends a username and password in the HTTP header, encoded using Base64.	❗ Easy to implement but not secure unless used over HTTPS. Susceptible to interception.
Bearer Authentication	Sends an access token (a string) in the HTTP header as proof of identity.	✅ More secure than basic auth. Common in OAuth flows. The token should be kept secret.
API Key	A unique identifier (string) provided to each client to access resources. Included in headers or query parameters.	⚠️ Simple to use but less secure. If exposed, anyone can use the key. Often rate-limited.
OAuth	A robust framework that uses both a password and a token for access. Supports scopes, refresh tokens, and expiration.	✅ Highly secure. Ideal for third-party integrations. Can be complex to implement initially.



NO HTTP

All authentication methods should be used over HTTPS to prevent credentials or tokens from being intercepted.

1.4 RESTful API Server Response

When a RESTful API server receives a valid request, it returns a response composed of three key parts:

Status Line

The **status line** includes a 3-digit HTTP status code that tells the client the result of the request.

Code Range	Meaning	Examples
2xx Success	The request was successfully processed.	200 OK, 201 Created
3xx Redirection	Further action is required to complete the request.	301 Moved Permanently
4xx Client Error	The request was incorrect or unauthorised.	400 Bad Request, 404 Not Found
5xx Server Error	The server encountered an error.	500 Internal Server Error

Message Body

The **message body** contains the data returned by the server, often in JSON format. **Example Response Body (JSON):**

```
{
  "id": 102,
  "name": "John",
  "age": 30,
  "email": "john@example.com"
}
```

! INFO

The format (JSON or XML) is usually negotiated via the request header (e.g., `Accept: application/json`).

Response Headers

Response headers provide metadata about the response. They're not part of the main data but give useful context.

Header	Description
Content-Type	Format of the response body (application/json)
Content-Length	Size of the body in bytes
Server	Info about the server software
Date	Timestamp of the response
Cache-Control	Whether/how long the response is cached
Access-Control-Allow-Origin	CORS setting for cross-origin requests

Example Full Response (Text Format)

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 67
Date: Tue, 22 Jul 2025 06:45:00 GMT
Server: nginx/1.18.0
```

```
{
  "id": 102,
  "name": "John",
  "age": 30,
  "email": "john@example.com"
}
```

Chapter 2. RESTful Architecture and Best Practices

2.1 REST as an Architectural Style

REST (Representational State Transfer) is not a protocol but an architectural style that guides the design of networked applications. It was introduced by **Roy Fielding** in his doctoral

dissertation to describe a set of architectural constraints that, when applied, enable scalable, reliable, and evolvable systems, especially in the context of web services.

These architectural constraints are not exclusive to HTTP or web APIs. Instead, they represent a model for designing distributed systems that emphasise clarity, separation of concerns, and stateless communication. RESTful APIs happen to be one concrete application of these principles over HTTP.

Architectural Constraints of REST

Constraint	Description
Stateless	Each client request must contain all the information necessary to understand and process it. The server should not store any client context between requests. This improves scalability and reliability.
Cacheable	Server responses should explicitly indicate whether they are cacheable. This enables clients and intermediaries to reuse responses and reduces the need for redundant requests.
Uniform Interface	REST enforces a standardised way to communicate between client and server. This includes using HTTP methods (GET, POST, PUT, DELETE), predictable URIs, and common data formats like JSON.
Client–Server Separation	The client and server operate independently. The client does not need to know about server implementation details, and the server does not manage client state. This separation allows for greater portability and scalability.
Layered System	REST allows intermediate layers (such as load balancers, proxies, or security filters) between client and server. These layers should not affect the request or response but can enhance performance, scalability, and security.
Code-on-Demand (optional)	Servers can temporarily extend client functionality by transferring executable code (e.g., JavaScript). This constraint is optional and less commonly used in RESTful API design.

2.2 RESTful API Design Principles

When we design a **RESTful API**, we are applying these architectural constraints to HTTP-based services. A good RESTful API is not just a CRUD wrapper over HTTP, it is a thoughtful, resource-oriented interface that adheres to REST principles to achieve robustness, scalability, and clarity.

! INFO

Follow the design principles of RESTful APIs to reduce redundancy and create a clear, interactive, and user-friendly interface.

“Non-CRUD” (Resource-Oriented) Actions

- **Instead of calling an action, create a resource:**

This means framing actions as resources. For example, rather than a URL like `/activateAccount`, which directly calls an action, you might use `/account-activation` which represents it as a resource.

- **Instead of “login”, create a “session”:**

This shifts the focus from the action of logging in to the creation of a session resource. So, instead of `/login`, you use `/sessions` to create a new session.

- **Instead of “closing account”, create a “closing”:**

This rephrases the action of closing an account as managing a resource, `/account-closings`. This might involve POSTing to this resource to request an account closure.

Aligning with HTTP Protocol Semantics

The HTTP protocol, which underlies web communications, is inherently designed to work with resources, identified by URIs (Uniform Resource Identifiers). HTTP methods (verbs) like GET, POST, PUT, DELETE are designed to perform standard operations on these resources.

Using these methods in a way that aligns with their intended use in the HTTP specification makes your API predictable and aligned with web standards, facilitating easier understanding and integration.

Simplifying and Standardising API Endpoints

When APIs focus on resources rather than actions, the number of endpoints typically reduces,

and their purpose becomes clearer:

- **Resource-oriented:**

An endpoint like `/sessions` handles all actions related to session management. Creating a session (logging in) might be a POST request to `/sessions`, and deleting a session (logging out) might be a DELETE request.

- **Action-oriented:**

Different actions might require different endpoints, like `/login` for logging in and `/logout` for logging out, even though they relate to the same conceptual resource (user sessions).

**TIP**

This approach reduces the cognitive load for developers who use your API, as they can often guess endpoint URLs based on standard practices and do not need to remember multiple endpoints for related actions.

2.3 API Contract for Collaboration

An **API contract** is essential because it defines the expectations and agreements between the API providers (backend) and its consumers (frontend). By having a clear contract, developers can build and integrate with the API more effectively, reducing the likelihood of misunderstandings and errors. It also serves as a reference point or verification for any updates or changes, ensuring that all parties are aligned on how the API should function.

**INFO**

See the [template on Canvas](#) to understand what an API contract should include.

2.4 REST API in Django

Traditional Django Views

- Traditional views in Django return `HttpResponse` objects, often rendering HTML.
- You can return JSON manually using `JsonResponse` and manually serialising data.

- However, this method:
 - Is not scalable or clean for building APIs.
 - Is more suitable for server-rendered web applications.
- Concerns to consider:
 - Business logic separation
 - Scalability
 - Performance optimisation

Need for Django REST Framework (DRF)

- In modern web applications (e.g., React SPAs, mobile apps), the backend often acts as an **API-only layer**.
- All frontend-backend communication happens via **JSON APIs**.
- This is known as a **headless architecture**, where frontend and backend are separated.
- **Django REST Framework (DRF)**:
 - Extends Django's capabilities to build Web APIs.
 - Optimised for JSON and other content types.
 - Supports:
 - Serialisation and deserialisation
 - Permissions and authentication
 - Many features out of the box



IN ASSIGNMENT

You are allowed to use raw `HttpResponse` and return HTML using Django's built-in view functions to meet the basic requirements. However, your project **must include API endpoints** that returns data in **JSON format**. This can be implemented either using simple `JsonResponse` objects or by creating a separate set of endpoints dedicated to JSON output.

While either approach is acceptable, we recommend using the **Django REST Framework** for a cleaner and more scalable API design. If you prefer a lightweight solution, you can use Django's `JsonResponse` to return JSON directly from any view, requiring only a small amount of code to serialise your data.



INFO

We will learn how to build RESTful APIs in Django using [Django REST Framework \(DRF\)](#) in the upcoming labs.

Chapter 3. GitHub Collaboration Practices

Git is a powerful version control system, but its real value and complexity become clear in team-based development. In this section, we introduce practical GitHub collaboration habits, not Git commands, that help you avoid conflicts, stay organised, and communicate effectively with your team.



TIP

You're not expected to master every Git command now. Instead, build the right habits so your future work becomes easier to manage and scale.

3.1 Getting Started with GitHub Tools

- [Git-SCM Book](#) – Comprehensive guide to Git.
- [GitHub Getting Started](#) – Official GitHub beginner guide.
- [GitHub Desktop](#) – A GUI client for GitHub that simplifies version control for beginners.



GitHub Desktop is beginner-friendly and includes visualisation tools to help you manage changes without using the command line.

3.2 Work Together with Pull Requests

Protect Important Branches

- Use **protected branches** (e.g., `main`) to prevent accidental changes.
- Set conditions like requiring CI tests or code reviews before merging.

Define Push Rules

- Set **rulesets** to control what gets pushed (e.g., file types, sizes).

- Useful for internal repositories or larger teams with stricter controls.

Resolve Merge Conflicts Before Merging

- GitHub **blocks merging** if there are unresolved conflicts between branches.
- Always check and resolve these using the file list shown above the merge button.

3.3 Track and Plan with GitHub Issues

Use **GitHub Issues** to:

- Track bugs, ideas, tasks, or any piece of work.
- Link issues to pull requests for visibility and context.
- Collaborate by assigning tasks to team members.

Helpful actions:

- [Link PR to issue](#)
- [Create branch for issue](#)
- [Assign issues & PRs](#)



TIP



Defining a clear **commit message guideline** and consistent **branch naming convention** can help your team better organise the repository and speed up reviews and debugging.



See further readings for suggested styles, or better yet, explore how mature open-source projects manage their GitHub repositories for real-world examples.



NO GITHUB ACTIONS ON USYD GITHUB ENTERPRISE

GitHub Actions is not enabled on the USYD GitHub Enterprise Server. ICT recommends using external CI/CD tools like Jenkins or AWS CodePipeline/CodeBuild instead.

Further Readings

RESTful

- What is a RESTful API?
- What Is a REST API? Examples, Uses, and Challenges
- Richardson, L., Amundsen, M., & Ruby, S. (2013). RESTful Web APIs. O'Reilly Media, Inc. ISBN 9781449359737.
- Fielding, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine, 2000.

Git Practice

- GitHub Docs
- Git Collaborating Workflows
- Commit Message Guidelines
- Naming conventions for Git Branches — a Cheatsheet

Credit

This lab content was authored and maintained by **Jiawen Wen**, with materials adapted from prior offerings and updated to align with 2026 delivery. All standards used in this lab (IEEE, ISO/IEC) are referenced for educational purposes under fair use and are available to enrolled students via the university's licensed repository.